

Raspberry Pi System Monitor in C++

A project-driven guide to Linux fundamentals, /proc, /sys, system calls, processes, networking, hardware telemetry, and terminal UI

Goal: build a lightweight Linux system monitor from scratch on a Raspberry Pi using C++, while understanding the operating-system mechanisms underneath.

Target: CPU usage, per-core usage, RAM/swap, load average, uptime, processes, disk usage, network traffic, temperature, and Raspberry Pi hardware information.

Core philosophy

- Use Linux interfaces such as /proc, /sys and POSIX APIs before reaching for monitoring libraries.
- Understand the source and meaning of every metric.
- Separate collection, calculation and presentation.
- Build a boring text version first; make it pretty later.

1. Linux Fundamentals You Need

Learn these concepts just deeply enough to understand what your C++ program is asking Linux to do.

Processes and PIDs

- A process is a running program identified by a PID.
- Linux exposes process information under `/proc/`.
- Learn parent/child processes, process states, users, virtual memory, file descriptors and signals.

Virtual filesystems

- **/proc**: kernel-maintained process and system information.
- **/sys**: kernel device, driver and hardware information.
- **/dev**: device nodes and hardware interfaces.

File descriptors and permissions

- Understand open/read/write/close conceptually.
- C++ streams eventually rely on OS I/O underneath.
- Learn users, groups, permissions, root and why some metrics may be inaccessible.

Time

- Understand wall-clock versus monotonic time.
- Use `std::chrono::steady_clock` for sampling intervals.
- Never rely on a clock that can jump when calculating rates.

2. Linux Interfaces for the Monitor

These are the main data sources and APIs you should master.

Interface	Metric	Concept
<code>/proc/stat</code>	CPU usage	Cumulative CPU jiffies
<code>/proc/meminfo</code>	RAM/swap	Memory counters
<code>/proc/loadavg</code>	Load	1/5/15 minute averages
<code>/proc/uptime</code>	Uptime	Elapsed time
<code>/proc/net/dev</code>	Network	RX/TX counters
<code>/proc/<PID>/...</code>	Processes	PID-specific state
<code>/sys/class/thermal/...</code>	Temperature	Thermal zones
<code>/sys/devices/...</code>	Hardware	Device/driver state
<code>statvfs()</code>	Disk	Filesystem capacity
<code>sysinfo()</code>	System	Uptime/memory/load
<code>uname()</code>	Kernel	Version/architecture

3. CPU Usage — Understand This Properly

Linux does not simply hand your program a fresh CPU percentage. `/proc/stat` exposes cumulative CPU time counters. You sample twice and calculate the difference.

Example

```
$ head -n 5 /proc/stat
cpu 12345 100 5432 987654 23 0 12 0 0 0
cpu0 3000 20 1300 240000 ...
cpu1 3100 30 1400 245000 ...
```

The fields represent user, nice, system, idle, iowait, irq, softirq, steal, guest and guest_nice. Start with the first four, then expand.

Calculation

```
total_delta = total_2 - total_1
idle_delta  = idle_2  - idle_1
busy_delta  = total_delta - idle_delta

cpu_usage = 100.0 * busy_delta / total_delta
```

If total time advances by 100 ticks and idle advances by 30, busy time is 70 ticks → 70% usage.

Per-core usage

Apply the same calculation to each `cpuN` line. Discover the number of cores dynamically instead of hardcoding it.

4. RAM and Swap

Read `/proc/meminfo`. Important fields include `MemTotal`, `MemAvailable`, `SwapTotal` and `SwapFree`.

```
MemTotal:      4048120 kB
MemAvailable:  2100000 kB
SwapTotal:     1048572 kB
SwapFree:      800000  kB
```

For user-facing RAM usage, `MemAvailable` is usually more meaningful than `MemFree` because Linux deliberately uses unused RAM for caches.

```
used = MemTotal - MemAvailable
ram_percent = 100.0 * used / MemTotal
```

Parse to bytes internally and convert to MiB/GiB only during formatting.

5. Load Average, Uptime and Host Info

Load average is not CPU percentage. `/proc/loadavg` reports 1-, 5- and 15-minute load averages. Keep the concepts separate in your UI.

For uptime, `/proc/uptime` provides elapsed seconds. You should also learn `sysinfo()`.

```
#include <sys/utsname.h>
#include <iostream>

int main() {
    struct utsname info{};
    if (uname(&info) == 0) {
        std::cout << info.sysname << '\n';
        std::cout << info.release << '\n';
        std::cout << info.machine << '\n';
    }
}
```

6. Process Monitoring

This is where the project becomes a serious Linux exercise.

- Enumerate numeric directories in `/proc`.
- Read `/proc//status` and `/proc//stat`.
- Collect PID, command, state, parent PID, RSS, virtual memory and CPU time.
- Expect processes to disappear between reads; this is normal.

Process enumeration algorithm

1. Iterate through `/proc`.
2. Keep numeric directory names.
3. Parse each PID.
4. Read PID-specific information.
5. Store `ProcessInfo` objects.
6. Sort by CPU%, memory%, or PID.
7. Display top N processes.

Per-process CPU

Process CPU time is also cumulative. Sample it twice, calculate its delta, and relate it to the system CPU-time delta and elapsed interval. Implement this only after total CPU usage works.

7. Disk Usage

Learn `statvfs()` instead of parsing the output of the `df` command.

```
#include <sys/statvfs.h>

struct statvfs fs{};
if (statvfs("/", &fs) == 0) {
    auto total = fs.f_blocks * fs.f_frsize;
    auto free  = fs.f_bfree  * fs.f_frsize;
    auto avail = fs.f_bavail * fs.f_frsize;
}
```

Understand total blocks, free blocks and blocks available to an unprivileged process.

8. Network Traffic

Read `/proc/net/dev`. RX and TX byte counts are cumulative counters.

```
rx_rate = (rx_bytes_2 - rx_bytes_1) / elapsed_seconds
tx_rate = (tx_bytes_2 - tx_bytes_1) / elapsed_seconds
```

This teaches a major systems-programming pattern: **cumulative counter** → **periodic sample** → **delta** → **rate**. CPU, network and many hardware metrics use the same idea.

9. Raspberry Pi Temperature and Hardware

Inspect your actual Pi rather than assuming one hardware path. Thermal zones are commonly exposed under `/sys/class/thermal/`.

```
ls /sys/class/thermal/
cat /sys/class/thermal/thermal_zone0/temp
```

A value such as 52000 commonly represents 52.000°C for a thermal-zone temperature file. Verify the semantics on your system.

Keep Raspberry-Pi-specific features in their own module so your core Linux monitor remains portable.

10. C++ Skills You Will Practice

- **std::ifstream**: read /proc and /sys.
- **std::filesystem**: iterate /proc and inspect paths.
- **std::chrono**: sampling intervals.
- **std::vector / unordered_map**: metric collections.
- **string/string_view**: parsing.
- **optional**: represent unavailable metrics.
- **error_code / exceptions**: robust failure handling.
- **Algorithms**: sorting processes and aggregating metrics.
- **Structs/classes**: clean data models.
- **RAII**: safe resource lifetime.

C++ ↔ Linux boundary

- **Standard C++**: streams, filesystem, chrono, containers.
- **POSIX/Linux APIs**: statvfs(), uname(), sysinfo(), gethostname().
- **Kernel interfaces**: /proc, /sys and device-specific files.

Do not avoid abstractions just to be 'low level'. The important skill is knowing what is underneath the abstraction.

11. Recommended Architecture

```
system-monitor/  
■■■ CMakeLists.txt  
■■■ README.md  
■■■ include/  
■   ■■■ cpu.hpp  
■   ■■■ memory.hpp  
■   ■■■ process.hpp  
■   ■■■ network.hpp  
■   ■■■ disk.hpp  
■   ■■■ temperature.hpp  
■   ■■■ system.hpp  
■■■ src/  
■   ■■■ main.cpp  
■   ■■■ cpu.cpp  
■   ■■■ memory.cpp  
■   ■■■ process.cpp  
■   ■■■ network.cpp  
■   ■■■ disk.cpp  
■   ■■■ temperature.cpp  
■   ■■■ system.cpp  
■■■ tests/  
■   ■■■ test_cpu.cpp  
■   ■■■ test_parsers.cpp
```

Keep the data model simple.

```
struct CpuSample {  
    unsigned long long user{};  
    unsigned long long nice{};  
    unsigned long long system{};  
    unsigned long long idle{};  
    unsigned long long iowait{};  
    unsigned long long irq{};  
    unsigned long long softirq{};  
    unsigned long long steal{};  
};  
  
struct CpuUsage {
```

```
double total_percent{};
std::vector<double> per_core_percent;
};
```

12. Build Roadmap

Stage 0 — Linux reconnaissance

Explore /proc, /sys, ps, top, free, uptime, df, ip and thermal files manually.

Stage 1 — System info

Implement hostname, kernel, architecture and uptime.

Stage 2 — RAM

Parse /proc/meminfo and calculate usage.

Stage 3 — CPU

Parse /proc/stat; calculate total and per-core usage.

Stage 4 — Refresh loop

Use steady_clock and refresh once per second.

Stage 5 — Temperature

Read thermal-zone data and handle missing paths.

Stage 6 — Disk

Use statvfs() for filesystem capacity.

Stage 7 — Network

Parse /proc/net/dev and calculate RX/TX rates.

Stage 8 — Processes

Enumerate /proc/PID and sort by memory.

Stage 9 — Process CPU

Calculate per-process CPU usage from cumulative counters.

Stage 10 — TUI

Only now add a terminal UI library.

Stage 11 — Logging

Export periodic metrics to CSV or JSON.

Stage 12 — Daemon

Learn signals, systemd and background operation.

13. Start With a Boring Terminal UI

```
CPU:  37.2%    RAM: 42.8%    TEMP: 51.2 C
LOAD: 0.74 0.62 0.51
```

```
NET:  RX 1.24 MB/s   TX 0.38 MB/s
DISK: /  61.4% used
```

PID	CPU%	MEM%	RSS	STATE	COMMAND
1021	28.4	4.2	170MB	S	python
732	8.1	1.3	52MB	S	sshd

Do not start with a dashboard library. First make the data layer correct. Then the UI becomes a separate engineering problem.

14. Error Handling and Testing

- Processes can vanish between two reads.
- Files can be inaccessible because of permissions.
- Kernel and hardware paths vary by system.
- Malformed input should not crash the monitor.
- Write parser tests using saved /proc-like text.
- Test CPU, RAM and network calculations with known samples.
- Compare your results with top, free, ps, ip and df under controlled loads.

Useful commands

```
cat /proc/stat
cat /proc/meminfo
cat /proc/loadavg
cat /proc/net/dev
cat /proc/<PID>/status
cat /proc/<PID>/stat
free -h
ps aux
df -h
uname -a
ls /sys/class/thermal/
```

15. Advanced Linux Topics After v1

- **Signals:** SIGINT/SIGTERM and graceful shutdown.
- **systemd:** run the monitor as a service.
- **Threads:** separate sampling and UI when needed.
- **epoll/select:** event-driven I/O concepts.
- **Scheduling:** nice values, priorities and CPU affinity.
- **cgroups:** resource accounting and limits.
- **Namespaces:** understand isolation and containers.
- **strace:** observe the system calls programs make.
- **perf:** later investigate performance profiling.

The strace exercise

```
strace -c cat /proc/meminfo
strace -e openat,read,close cat /proc/meminfo
strace -e statfs,statx df /
```

The goal is to connect your C++ code to the OS boundary: user-space program → library/API → kernel → resource.

16. Final Project Specification

The finished v1 should run locally on the Raspberry Pi with no Python, web server or cloud dependency.

```
Raspberry Pi System Monitor
-----
Host: raspberrypi
Kernel: Linux 6.x
Uptime: 3d 04h 17m

CPU      31.7%
CPU0     28.2%
CPU1     35.1%
CPU2     29.8%
CPU3     33.7%

Memory   1.72 / 3.83 GiB   44.9%
Swap     0.00 / 1.00 GiB   0.0%
Load     0.71 0.62 0.54

Temp     51.4 C
Disk /   62.1%   28.4 / 45.7 GiB

Network
eth0     RX 1.20 MB/s   TX 0.34 MB/s

Top processes
PID      CPU%    MEM%    RSS      COMMAND
...
```

Optional v2: process interaction, sorting, history graphs, CSV/JSON logging, configurable refresh rate, systemd service mode and Raspberry-Pi-specific throttling/thermal information.

17. Learning Checklist

- Understand processes, PIDs and file descriptors.
- Navigate /proc and /sys manually.
- Parse Linux virtual files safely in C++.
- Understand cumulative counters versus rates.
- Calculate CPU usage from two /proc/stat samples.
- Calculate RAM usage from /proc/meminfo.
- Calculate network throughput from two samples.
- Enumerate /proc/PID and handle disappearing processes.
- Use statvfs(), uname() and basic POSIX APIs.
- Use std::chrono correctly.
- Handle permissions and missing files.
- Test parsers separately from the UI.
- Use strace to inspect OS interactions.
- Build and run the monitor on the Raspberry Pi.

18. Study Order

```
Linux basics
↓
/proc + /sys exploration
↓
C++ file parsing
↓
CPU + RAM
↓
```


chrono + sampling
↓
temperature + disk
↓
network counters
↓
process enumeration
↓
per-process CPU/RAM
↓
POSIX APIs
↓
signals + systemd
↓
strace + deeper Linux
↓
TUI / polish

Core principle: first understand where Linux gets the number, then write the smallest C++ code that can read and calculate it, and only then make the interface pretty.